

Scientific Calculator

EE24BTECH11049

Patnam Shariq Faraz Muhammed

Abstract

This report presents a comprehensive analysis of our Arduino-based scientific calculator implementation. The calculator utilizes the Shunting Yard algorithm for efficient expression parsing and employs RK4 for custom mathematical function calculations. We explore the electronic circuit design and software architecture, emphasizing the expression evaluation process, optimized mathematical computations, and an intuitive user interface.

1	Introduction	3
2	Hardware Components	3
3	Connections	3
4	Mathematical Expressions	5
4.1	Core Trigonometric Functions	5
4.2	Inverse Trigonometric Functions	5
4.3	Exponential and Logarithmic Functions	5
4.4	Hyperbolic Functions	5
4.5	Utility Functions	5
4.6	Mathematical Constants	5
4.7	Important Implementation Notes	5
4.8	Code-funcs.c	6
5	Overall System Architecture	10
6	Expression Parsing and Evaluation System	10
6.1	Key Data Structures	10
6.2	Shunting Yard Algorithm for Parsing	10
6.3	Evaluation of RPN Expressions	11
7	User Interface and Input Handling	11
7.1	Input Mechanisms	11
7.2	Button Matrix Scanning and Debouncing	12
7.3	Debouncing Mechanism	12
7.4	Display Management	12
8	Hardware Interaction Layer	13
8.1	Memory Management	13
8.2	Microcontroller Interfaces	13
9	Optimization Strategies	13
9.1	Memory Optimization	13
9.2	Performance Optimization	13
10	Key Design Patterns	13
11	Potential Improvements	13

1 INTRODUCTION

The calculator project implements a functional and extendable scientific calculator capable of evaluating complex mathematical expressions with proper operator precedence. Key features include:

The calculator utilizes the Shunting Yard algorithm for expression evaluation and implements mathematical functions using RK4 method and numerical approximations. By avoiding external libraries, it ensures precise calculations while maintaining a minimal memory footprint, making it well-suited for the constrained AVR microcontroller environment.

2 HARDWARE COMPONENTS

The following are components required for the project

- Arduino UNO
- Breadboard
- 36 push buttons
- Potentiometer
- Jumper wires
- Cell phone (to power the Arduino)

3 CONNECTIONS

Signal/Pin Name	Arduino Connection
LCD Display	
LCD_E (Enable)	PB1
DB4 (Data Bit 4)	Pin 2
DB5 (Data Bit 5)	Pin 3
DB6 (Data Bit 6)	Pin 4
DB7 (Data Bit 7)	Pin 5
Push Button	
ROW signals	PORTC (DDR: DDRC, PIN: PINC)
COLUMN signals	PORTD (DDR: DDRD, PIN: PIND)
Number of buttons	36

TABLE 0: Arduino and LCD Push Button Connections

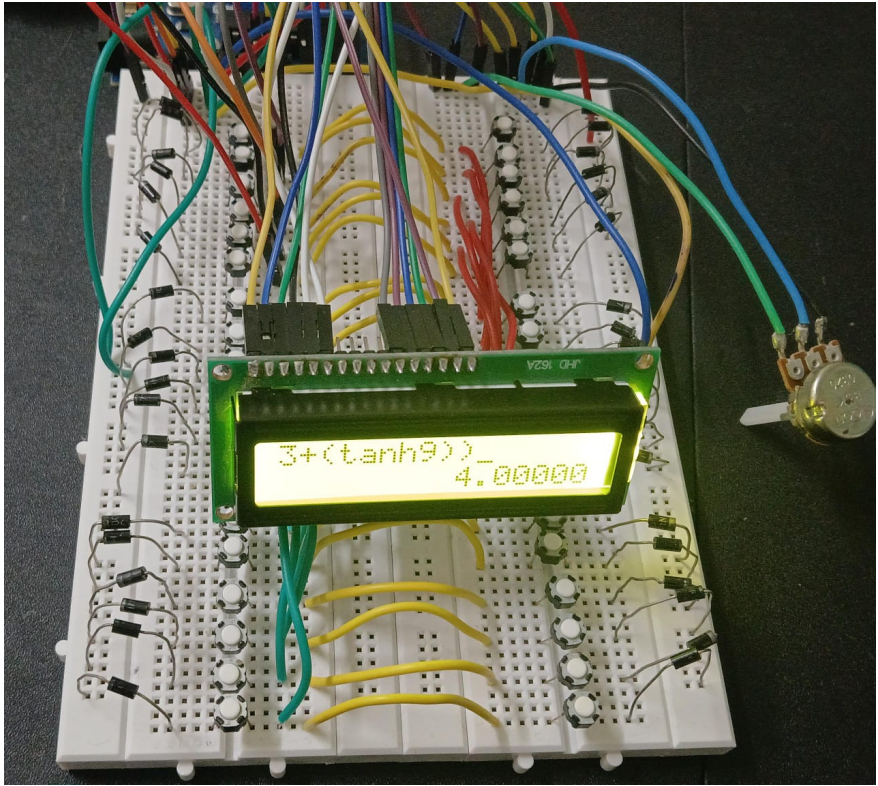


Fig. 0.1: Calculator

4 MATHEMATICAL EXPRESSIONS

4.1 Core Trigonometric Functions

- $\sin(x)$, $\cos(x)$, $\tan(x)$: Implemented using RK4 to solve the differential equation $y'' = -y$ with appropriate initial conditions. These provide the fundamental trigonometric operations.

4.2 Inverse Trigonometric Functions

- $\arcsin(x)$, $\arccos(x)$, $\arctan(x)$: Calculate inverse trigonometric functions using RK4 with their respective differential equations.

4.3 Exponential and Logarithmic Functions

- $\text{pow}(x, w)$: Power function using RK4 for $\frac{dy}{dx} = w \frac{y}{x}$
- $\ln(x)$: Natural logarithm using RK4 for $\frac{dy}{dx} = \frac{1}{x}$

4.4 Hyperbolic Functions

- $\sinh(x)$, $\cosh(x)$, $\tanh(x)$: Hyperbolic functions implemented using the exponential definitions.

4.5 Utility Functions

- $\text{fast_inv_sqrt}(x)$: Fast inverse square root using the famous Quake III algorithm.
- $\text{principal_range}(\theta)$: Normalizes angles to $[0, 2\pi]$ range.
- $\text{factorial}(n)$: Calculates factorial for non-negative integers.
- $\text{dtf}(\text{decimal}, * \text{numerator}, * \text{denominator})$: Converts decimals to fractions.

4.6 Mathematical Constants

- $\pi = 3.14159265358979323846$
- $e = 2.7182818284$

4.7 Important Implementation Notes

- 1) The functions use a step size $H = 0.01$ for numerical approximation.
- 2) Error handling is implemented for edge cases (negatives, zeros).
- 3) The code uses fast_inv_sqrt for optimization where appropriate.

For a calculator implementation, these functions provide a complete set of mathematical operations covering:

- Basic arithmetic (through pow)
- Trigonometry (standard and inverse functions)
- Exponential and logarithmic calculations
- Hyperbolic functions
- Number theory operations (GCD, factorial, decimal-to-fraction conversion)

The numerical approach using RK4 is particularly interesting as it solves the functions through differential equations rather than series expansions, potentially offering good performance for a wide range of inputs.

4.8 Code-funcs.c

```

1  #include <stdint.h>
2
3
4  #define PI 3.14159265358979323846
5  #define H 0.01
6  #define MAX_ITERS 100000
7  #define E 2.7182818284
8
9  double fast_inv_sqrt(double x){
10     if(x <= 0) return 0;
11     x = (float) x;
12     long i;
13     float x2, y;
14     const float threehalfs = 1.5F;
15
16     x2 = x * 0.5F;
17     y = x;
18     i = * ( long * ) &y;
19     i = 0x5f3759df - ( i >> 1 );
20     y = * ( float * ) &i;
21     y = y * ( threehalfs - ( x2 * y * y ) );
22
23     return (double) y;
24 }
25
26 double principal_range(double angle) {
27     int k = (int)(angle / (2*PI));
28     angle -= k * (2*PI);
29     if (angle < 0) angle += (2*PI);
30     return angle;
31 }
32
33 // Sine calculation using RK4 for y'' = -y
34 double sin(double x_target) {
35     double x = 0.0; // Start point
36     double y = 0.0; // y(0) = 0
37     double dy = 1.0; // y'(0) = 1
38
39     double h = H; // Step size
40     int steps = (int)(x_target / h); // Number of steps
41
42     for (int i = 0; i < steps; i++) {
43         double k1_y = h * dy;
44         double k1_dy = h * (-y);
45
46         double k2_y = h * (dy + 0.5 * k1_dy);
47         double k2_dy = h * (-(y + 0.5 * k1_y));
48
49         double k3_y = h * (dy + 0.5 * k2_dy);
50         double k3_dy = h * (-(y + 0.5 * k2_y));
51
52         double k4_y = h * (dy + k3_dy);
53         double k4_dy = h * (-(y + k3_y));
54
55         y += (k1_y + 2 * k2_y + 2 * k3_y + k4_y) / 6.0;
56         dy += (k1_dy + 2 * k2_dy + 2 * k3_dy + k4_dy) / 6.0;
57

```

```

58     x += h;
59 }
60
61 return y; // Return y at x_target
62 }
63
64 double cos(double x_target) {
65     double x = 0.0; // Start point
66     double y = 1.0; // y(0) = 1
67     double dy = 0.0; // y'(0) = 0
68
69     double h = H; // Step size
70     int steps = (int)(x_target / h); // Number of steps
71
72     for (int i = 0; i < steps; i++) {
73         double k1_y = h * dy;
74         double k1_dy = h * (-y);
75
76         double k2_y = h * (dy + 0.5 * k1_dy);
77         double k2_dy = h * (-(y + 0.5 * k1_y));
78
79         double k3_y = h * (dy + 0.5 * k2_dy);
80         double k3_dy = h * (-(y + 0.5 * k2_y));
81
82         double k4_y = h * (dy + k3_dy);
83         double k4_dy = h * (-(y + k3_y));
84
85         y += (k1_y + 2 * k2_y + 2 * k3_y + k4_y) / 6.0;
86         dy += (k1_dy + 2 * k2_dy + 2 * k3_dy + k4_dy) / 6.0;
87
88         x += h;
89     }
90
91     return y; // Return y at x_target
92 }
93
94 double tan(double x){
95     return sin(x)/cos(x);
96 }
97
98 // Power function using RK4 for dy/dx = w*y/x
99 double pow(double x, double w) {
100     if(x < 0 && ((int) w) != w) return 0;
101     if (x == 0) return 0;
102     if (w == 0) return 1;
103
104     if(x < 0) return ( ((int) w)%2 == 0 ? 1: -1)*pow(-x, w);
105
106     double x0 = 1.0, y = 1.0;
107     double target = x;
108     int steps = (int)((target - x0) / H);
109
110     for (int i = 0; i < steps; i++) {
111         double k1 = H * w * y / x0;
112         double k2 = H * w * (y + k1/2) / (x0 + H/2);
113         double k3 = H * w * (y + k2/2) / (x0 + H/2);
114         double k4 = H * w * (y + k3) / (x0 + H);
115
116         y += (k1 + 2*k2 + 2*k3 + k4) / 6;

```

```

117     x0 += H;
118 }
119
120 return y;
121 }
122
123 // Natural log using RK4 for dy/dx = 1/x
124 double ln(double x) {
125     if (x <= 0) return 0.0;
126     if (x < 1) return -ln(1/x);
127
128     double x0 = 1.0, y = 0.0;
129     int steps = (int)((x - x0) / H);
130
131     for (int i = 0; i < steps; i++) {
132         double k1 = H / x0;
133         double k2 = H / (x0 + H/2);
134         double k3 = H / (x0 + H/2);
135         double k4 = H / (x0 + H);
136
137         y += (k1 + 2*k2 + 2*k3 + k4) / 6;
138         x0 += H;
139     }
140
141     return y;
142 }
143
144 double arctan(double x) {
145     double x0 = 0.0, y = 0.0;
146     int steps = x >= 0 ? (int) (x / H): (int) (-x / H);
147     double step_dir = (x >= 0) ? 1 : -1;
148
149     for (int i = 0; i < steps; i++) {
150         double k1 = H / (1 + x0*x0);
151         double k2 = H / (1 + (x0 + H/2)*(x0 + H/2));
152         double k3 = H / (1 + (x0 + H/2)*(x0 + H/2));
153         double k4 = H / (1 + (x0 + H)*(x0 + H));
154
155         y += step_dir * (k1 + 2*k2 + 2*k3 + k4) / 6;
156         x0 += step_dir * H;
157     }
158
159     return y;
160 }
161
162 double arcsin(double x) {
163     if (x < -1 || x > 1) return 0;
164
165     double x0 = 0.0, y = 0.0;
166     int steps = x >= 0 ? (int) (x / H): (int) (-x / H);
167     double step_dir = (x >= 0) ? 1 : -1;
168
169     for (int i = 0; i < steps; i++) {
170         double k1 = H * fast_inv_sqrt(1 - x0*x0);
171         double k2 = H * fast_inv_sqrt(1 - (x0 + step_dir*H/2)*(x0 + step_dir*H/2));
172         double k3 = H * fast_inv_sqrt(1 - (x0 + step_dir*H/2)*(x0 + step_dir*H/2));
173         double k4 = H * fast_inv_sqrt(1 - (x0 + step_dir*H)*(x0 + step_dir*H));
174
175         y += step_dir * (k1 + 2*k2 + 2*k3 + k4) / 6;

```



```

176     x0 += step_dir * H;
177 }
178
179 return y;
180 }
181
182 double arccos(double x){
183     return ((PI/2) - arcsin(x));
184 }
185
186 double factorial(double n) {
187     if(n < 0) return 0;
188     if(n == 0 || n == 1) return 1;
189
190     double result = 1.0;
191
192     for(double i = (int) n; i >= 2 ; i--) {
193         result *= i;
194     }
195
196     return result;
197 }
198
199 int gcd(int a, int b) {
200     int temp;
201
202     while (b != 0) {
203         temp = b;
204         b = a % b;
205         a = temp;
206     }
207
208     return a;
209 }
210
211 void dtf(double decimal, long int* numerator, long int* denominator) {
212     *numerator = (int)(decimal * 1000000);
213     *denominator = 1000000;
214
215     int gcd_ = gcd(*numerator, *denominator);
216
217     *numerator = *numerator/gcd_;
218     *denominator = *denominator/gcd_;
219 }
220
221 double sinh(double x) {
222     return (pow(E, x) - pow(E, -x)) / 2;
223 }
224
225 double cosh(double x) {
226     return (pow(E, x) + pow(E, -x)) / 2;
227 }
228
229 double tanh(double x) {
230     return sinh(x) / cosh(x);
231 }

```

5 OVERALL SYSTEM ARCHITECTURE

The embedded calculator is implemented on an AVR microcontroller and consists of three main components:

- Expression Parsing and Evaluation System
- User Interface and Input Handling
- Hardware Interaction Layer

6 EXPRESSION PARSING AND EVALUATION SYSTEM

6.1 Key Data Structures

The token structure represents different types of mathematical elements:

```
1 typedef struct Token {
2     TokenType type; // Type of token (number, operator, function)
3     TokenVal val; // Value of the token
4 } Token;
```

Listing 1: Token Structure

6.2 Shunting Yard Algorithm for Parsing

The calculator uses the Shunting Yard algorithm to convert infix expressions to Reverse Polish Notation (RPN). This algorithm ensures operator precedence and left-to-right evaluation.

The algorithm is implemented as follows:

```
1 void processTokens(Token token_stream[], short size, Token output_stack[], short
   *output_size, Token operator_stack[], short *operator_size) {
2     for (int i = 0; i < size; i++) {
3         Token token = token_stream[i];
4
5         if (token.type == NUM) {
6             append(output_stack, output_size, token);
7         } else if (token.type == OP) {
8             while (*operator_size > 0 && precedence(token.val.op) <=
               precedence(operator_stack[*operator_size - 1].val.op)) {
9                 append(output_stack, output_size, pop(operator_stack,
               operator_size));
10            }
11            append(operator_stack, operator_size, token);
12        } else if (token.type == LBRAK) {
13            append(operator_stack, operator_size, token);
14        } else if (token.type == RBRAK) {
15            while (*operator_size > 0 && operator_stack[*operator_size - 1].type !=
               LBRAK) {
16                append(output_stack, output_size, pop(operator_stack,
               operator_size));
17            }
18            pop(operator_stack, operator_size); // Remove left bracket
19        }
20    }
21
22    while (*operator_size > 0) {
23        append(output_stack, output_size, pop(operator_stack, operator_size));
```

```

24     }
25 }

```

Listing 2: Shunting Yard Algorithm Implementation

6.3 Evaluation of RPN Expressions

Once the expression is converted to RPN, it is evaluated using a stack-based approach.

```

1  double evaluatorRPN(Token output_stack[], short output_size, double ans) {
2      Token res_stack[STACK_SIZE];
3      short res_size = 0;
4
5      for (int i = 0; i < output_size; i++) {
6          Token token = output_stack[i];
7
8          if (token.type == NUM) {
9              append(res_stack, &res_size, token);
10         } else if (token.type == OP) {
11             Token right = pop(res_stack, &res_size);
12             Token left = pop(res_stack, &res_size);
13             Token res_token;
14             res_token.type = NUM;
15
16             switch (token.val.op) {
17                 case ADD: res_token.val.num = left.val.num + right.val.num; break;
18                 case SUB: res_token.val.num = left.val.num - right.val.num; break;
19                 case MUL: res_token.val.num = left.val.num * right.val.num; break;
20                 case DIV: res_token.val.num = left.val.num / right.val.num; break;
21                 case POW: res_token.val.num = pow(left.val.num, right.val.num);
22                     break;
23             }
24             append(res_stack, &res_size, res_token);
25         }
26     }
27     return res_stack[0].val.num;
28 }

```

Listing 3: RPN Evaluation

7 USER INTERFACE AND INPUT HANDLING

7.1 Input Mechanisms

The calculator employs a 6x6 matrix keypad with the following features:

- Mode switching capability
- Button debouncing mechanism
- Two button mapping arrays for Standard and Advanced modes

The keypad handling function is implemented as follows:

```

1  char getKeypadInput() {
2      for (int row = 0; row < ROWS; row++) {
3          for (int col = 0; col < COLS; col++) {
4              if (isButtonPressed(row, col)) {
5                  return keyMap[row][col];
6              }
9         }

```

```

7     }
8     }
9     return '\0';
10  }

```

Listing 4: Keypad Input Handling

7.2 Button Matrix Scanning and Debouncing

A button matrix scanning mechanism ensures accurate key detection. The process involves:

- Setting each row LOW sequentially
- Reading column states to detect key presses
- Implementing software debouncing for stable key readings

The button matrix scanning function is implemented as follows:

```

1  void scanButtonMatrix() {
2      for (int row = 0; row < ROWS; row++) {
3          setRowLow(row);
4
5          for (int col = 0; col < COLS; col++) {
6              if (readColumn(col) == LOW) {
7                  debounceButton(row, col);
8              }
9          }
10
11         setRowHigh(row);
12     }
13 }

```

Listing 5: Button Matrix Scanning

7.3 Debouncing Mechanism

Debouncing prevents multiple unwanted detections from a single press:

```

1  void debounceButton(int row, int col) {
2      _delay_ms(DEBOUNCE_TIME);
3      if (readColumn(col) == LOW) {
4          registerKeyPress(row, col);
5      }
6  }

```

Listing 6: Debounce Function

7.4 Display Management

The LCD display:

- Supports a 16x2 character display
- Handles multi-line expressions
- Converts abbreviated function names to full names
- Manages scrolling for long expressions

8 HARDWARE INTERACTION LAYER

8.1 Memory Management

- Uses EEPROM for persistent storage
- Supports memory recall and storage operations
- Employs optimized memory management using fixed-size buffers

The EEPROM read function is shown below:

```

1 uint8_t readEEPROM(uint16_t address) {
2     while (EECR & (1 << EEPF)); // Wait for completion
3     EEAR = address;
4     EECR |= (1 << EERE);
5     return EEDR;
6 }
```

Listing 7: EEPROM Read Function

8.2 Microcontroller Interfaces

- Direct port manipulation for LCD and keypad
- Uses AVR-specific libraries and macros
- Minimizes dynamic memory allocation

9 OPTIMIZATION STRATEGIES

9.1 Memory Optimization

- Fixed-size statically allocated arrays
- Minimal dynamic memory usage
- Efficient bit manipulation macros
- Compact token representation using a union

9.2 Performance Optimization

- Efficient parsing algorithm
- Minimal computational overhead
- Direct hardware register manipulation
- Predefined constants for repeated calculations

10 KEY DESIGN PATTERNS

The software architecture employs:

- **State Machine:** Used for button handling
- **Interpreter Pattern:** Applied in expression evaluation
- **Command Pattern:** Used for button mapping

11 POTENTIAL IMPROVEMENTS

Future enhancements may include:

- Advanced error handling for complex expressions
- Additional mathematical functions
- Improved memory management
- Enhanced floating-point precision handling